Course Folder Management System

Server Deployment Guide (self-hosted, on-premise) - end to end

University Institute of Information Technology (UIIT), PMAS Arid Agriculture University, Rawalpindi

Prepared by: Noman Amjad

This guide deploys the system on your own Linux server using Docker. Everything runs on the one server: the web application, the PostgreSQL database, and the uploaded files. No cloud account is required. Follow the steps in order; by the end the system is reachable in a browser and the focal person can log in.

What you need before starting: (1) a server running **Ubuntu Server 22.04 or 24.04** (a physical machine or a virtual machine) that you can log into; (2) administrator (sudo) access on it; (3) the server's IP address on your network (for example 192.168.1.50); (4) access to the private GitHub repository (a GitHub account that can read it).

1. Log in to the server

From your own computer, open a terminal (on Windows use PowerShell) and connect over SSH. Replace the user and IP with yours:

```
ssh youruser@192.168.1.50
```

Enter the password when asked. You are now on the server. All commands below are run on the server unless stated otherwise.

2. Install Docker and Docker Compose

Update the system and install Docker using the official script, then add your user to the docker group so you do not need sudo each time:

```
sudo apt update && sudo apt -y upgrade curl -fsSL https://get.docker.com | sudo sh sudo usermod -aG docker $USER
newgrp docker docker --version docker compose version
```

If the last two commands print version numbers, Docker is ready. (Docker Compose v2 is included with Docker; you call it as `docker compose`.)

3. Get the project onto the server

The repository is private, so cloning needs a GitHub **Personal Access Token (PAT)** instead of your normal password.

3.1 Create a token (once, in your browser on any computer)

1. Go to GitHub - Settings - Developer settings - Personal access tokens - Tokens (classic) - Generate new token (classic).
2. Give it a name, set an expiry, tick the **repo** scope, and generate it. Copy the token (it is shown once).

3.2 Clone on the server

```
sudo apt -y install git git clone https://github.com/780Noman/course-folder-management-system.git # Username:
your GitHub username # Password: paste the Personal Access Token (not your account password) cd
course-folder-management-system
```

Alternative without git: on GitHub click **Code - Download ZIP**, then copy it to the server with `scp course-folder-management-system.zip youruser@192.168.1.50:~`, and on the server run `sudo apt -y install unzip && unzip course-folder-management-system.zip`.

4. Create the configuration file (.env)

The application reads all secrets and settings from a file named `.env`. Start from the example and edit it:

```
cp .env.example .env nano .env
```

Set these values (leave the rest as they are). Use your server's IP address.

Setting	Value to put
`DJANGO_SETTINGS_MODULE`	`config.settings.prod`
`SECRET_KEY`	a long random string (generate it in step 4.1)
`DEBUG`	`False`
`ALLOWED_HOSTS`	your server IP, e.g. `192.168.1.50` (add a domain too if you have one, comma-separated)
`DATABASE_URL`	`postgres://postgres:CHANGE_DB_PASSWORD@db:5432/coursefolders`
`POSTGRES_PASSWORD`	the same `CHANGE_DB_PASSWORD` you used above
`ENABLE_HTTPS`	`False` (internal HTTP access). Set `True` only if you put it behind HTTPS.
`CSRF_TRUSTED_ORIGINS`	`http://192.168.1.50:8000` (must match how users open it)
`PDF_ENGINE`	`weasyprint` (the server image includes it)

Leave the `AWS_*` storage values **empty** - this makes the system store uploaded files on the server itself (kept safe in a Docker volume). Leave the `EMAIL_*` values empty for now; invitations can be set without email (see step 8). In nano, save with Ctrl+O then Enter, and exit with Ctrl+X.

4.1 Generate the SECRET_KEY

```
docker run --rm python:3.12-slim python -c "import secrets; print(secrets.token_urlsafe(50))"
```

Copy the printed line into `SECRET_KEY` in `.env`.

Important: never share or commit the `.env` file. It contains your secret key and database password.

5. Build and start the system

```
docker compose -f docker-compose.prod.yml up -d --build
```

The first run downloads and builds everything (a few minutes). It starts two containers: the database and the web application. The web container automatically applies the database migrations and collects static files on start.

Check that both are running and watch the logs:

```
docker compose -f docker-compose.prod.yml ps docker compose -f docker-compose.prod.yml logs -f web
```

When the log shows Gunicorn has started (workers booted), press Ctrl+C to stop watching the log (this does not stop the server).

6. Open the firewall (if enabled)

If the Ubuntu firewall is on, allow the application port:

```
sudo ufw allow 8000/tcp
```

7. Open the system in a browser

On any computer on the same network, open:

```
http://192.168.1.50:8000
```

You should see the home page with a Sign in button. (If it does not load, see the Troubleshooting section.)

8. Create the first administrator (focal person)

This is the one account you create by hand; it has full access and then invites everyone else. Run:

```
docker compose -f docker-compose.prod.yml run --rm web python manage.py createsuperuser
```

Enter an email (for example `focal@uiit.edu.pk`), a name, and a password. For now use a simple but valid password such as `Uiit@2026` (at least 8 characters, not all numbers). The focal person can change it after first login from the navbar link **Change password**.

9. First login and smoke test

1. Open `http://192.168.1.50:8000` and sign in as the focal person.
2. Create a **Term** and mark it current; create a **Course** and assign an instructor.
3. Open **Manage faculty - Add faculty** to invite a teacher (see the email note below).
4. As that faculty member, open the course folder, upload a file, and submit the mid-term; as the focal person, review and approve, then issue the certificate.

About invitations/email: if the `EMAIL_*` settings are empty, the invitation link is written to the server log instead of being emailed. To read it after clicking "Send invitation", run `docker compose -f docker-compose.prod.yml logs --tail=50 web` and copy the link. To send real emails later, fill the `EMAIL_*` values in `.env` (university SMTP or a free provider) and restart with the command in step 11.

10. Day-to-day operations

Task	Command (run inside the project folder)
See running containers	`docker compose -f docker-compose.prod.yml ps`
View logs	`docker compose -f docker-compose.prod.yml logs -f web`
Stop the system	`docker compose -f docker-compose.prod.yml down`
Start it again	`docker compose -f docker-compose.prod.yml up -d`
Create another admin	`docker compose -f docker-compose.prod.yml run --rm web python manage.py createsuperuser`

The system restarts automatically if the server reboots (the containers are set to restart unless stopped).

11. Updating to a new version

```
cd course-folder-management-system git pull docker compose -f docker-compose.prod.yml up -d --build
```

Migrations and static files are handled automatically on start. Your data (database and uploaded files) is preserved in Docker volumes and is not affected by rebuilds.

12. Backups (do this regularly)

Two things must be backed up: the database and the uploaded files.

```
# Database backup (writes a dated file to your home folder) docker compose -f docker-compose.prod.yml exec -T db \ pg_dump -U postgres coursefolders > ~/coursefolders-$(date +%F).sql # Uploaded files backup (copies the media volume to a tar file) docker run --rm -v course-folder-management-system_media:/m -v $HOME:/out \ busybox tar czf /out/media-$(date +%F).tar.gz -C /m .
```

Copy these backup files off the server (to another disk or location). To restore, load the .sql with `psql` and extract the media tar back into the volume. See `docs/BACKUPS.md` in the project for details.

13. Troubleshooting

Symptom	What to do
Page does not open in the browser	Check containers are up (step 10), and the firewall allows port 8000 (step 6). Confirm you used `http://` and the correct IP and port.
"Bad Request (400)"	The server IP/domain is not in `ALLOWED_HOSTS` in `.env`. Add it, then restart (step 11, the rebuild line).
Login or buttons fail with a CSRF/403 error	Set `CSRF_TRUSTED_ORIGINS` to exactly how users open the site, e.g. `http://192.168.1.50:8000`, and keep `ENABLE_HTTPS=False` for plain HTTP. Restart.
It redirects to https and won't load	`ENABLE_HTTPS` must be `False` for plain HTTP access. Fix it in `.env` and restart.
Database errors on start	Make sure `POSTGRES_PASSWORD` and the password inside `DATABASE_URL` match. Then `down` and `up` again.

14. Optional: HTTPS and a friendly address

For access by a name (e.g. https://folders.uiit.edu.pk) with a padlock, place a reverse proxy (Caddy or Nginx) in front that terminates TLS and forwards to port 8000, then set `ENABLE_HTTPS=True` and add the domain to `ALLOWED_HOSTS` and `CSRF_TRUSTED_ORIGINS`. This needs a DNS name pointing at the server and is best done with help from the network administrator. It is not required for internal use by IP.